UMRR9F T169 MSE V1.1.0 USER INTERFACE

DATE:
February 16, 2026

USER INTERFACE NAME:
UMRR9F T169 MSE

USER INTERFACE VERSION:
v1.1.0

s.m.s, smart microwave sensors GmbH
In den Waashainen 1
38108 Braunschweig
Germany

Phone: +49 531 39023-0
Fax: +49 531 39023-599
info@smartmicro.de
www.smartmicro.com

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

CONTENT

1 COMMUNICATION DATA STREAM SERVICE 2
1.1 COMTARGETLIST PORT 2
1.2 COMOBJECTLIST PORT 4
1.3 COMOBJECTBASELIST PORT 6
1.4 COMTARGETBASELIST PORT 8
2 LEGAL DISCLAIMER NOTICE 10
APPENDIX 11
A COMMUNICATION DATA SERVICE API 11
B USER INTERFACE INSTRUCTIONS UMRR9F T169 MSE VERSION 1.1.0 24
B.1 Parameter Section auto_interface_0dim 24
B.2 Status Section auto_interface 28
B.3 Command Section auto_interface_command 30

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

1 COMMUNICATION DATA STREAM SERVICE

With the communication data stream service smartmicro ports can be received as C++ objects with simplified access functions, which are generated by the user interface. Smartmicro ports are data buffers which contains data recorded by the radar data: e.g objects, statistics, statuses of device etc. Each port contains a generic port header, with a port description : version, id, size etc. Sometime ports also contains dynamic list of objects. In order to receive a port, a callback needs to be registered with the service. The callback will be carried out periodically every sensor cycle time.

Please note:

- This callback will be called in the context of a receiver thread, so the data needs to be copied and the callback must be released. Otherwise, the reception will be blocked.
- It is possible to use one callback function for several clients with the same port and same user interface, but it is not allowed to use one callback function for different ports or different user interfaces.

For more details please see the examples below. The following ports are supported.

1.1 COMTARGETLIST PORT

Description:
To receive the port called "ComTargetList" from a specific client, please use the following registration interface:

```
#include <umrr9f_t169_mse_v1_1_0/DataStreamServiceIface.h>

void ReceiveComTargetListClbk(IN std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::
    comtargetlist::ComTargetList> comTargetList, com::types::ClientId clientId)
{
        // Getting members of  ComTargetList
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comtargetlist::PortHeader> portHeader =
                        comTargetList->GetPortHeader();
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comtargetlist::TargetListHeader>
        targetListHeader =
                        comTargetList->GetTargetListHeader();
    auto targetList = comTargetList->GetTargetList();
    // Getting members of  PortHeader
    std::cout << "Variable PortIdentifier:"
              << portHeader->GetPortIdentifier()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << portHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << portHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << portHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << portHeader->GetPortSize()
              << std::endl;
    std::cout << "Variable BodyEndianness:"
              << portHeader->GetBodyEndianness()
              << std::endl;
    std::cout << "Variable PortIndex:"
              << portHeader->GetPortIndex()
              << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
              << portHeader->GetHeaderVersionMajor()
              << std::endl;
    std::cout << "Variable HeaderVersionMinor:"
              << portHeader->GetHeaderVersionMinor()
              << std::endl;
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
// Getting members of  TargetListHeader
std::cout << "Variable CycleTime:"
          << targetListHeader->GetCycleTime()
          << std::endl;
std::cout << "Variable NumberOfTargets:"
          << targetListHeader->GetNumberOfTargets()
          << std::endl;
std::cout << "Variable AcquisitionTxAntIdx:"
          << targetListHeader->GetAcquisitionTxAntIdx()
          << std::endl;
std::cout << "Variable AcquisitionSweepIdx:"
          << targetListHeader->GetAcquisitionSweepIdx()
          << std::endl;
std::cout << "Variable AcquisitionCfIdx:"
          << targetListHeader->GetAcquisitionCfIdx()
          << std::endl;
std::cout << "Variable Prf:"
          << targetListHeader->GetPrf()
          << std::endl;
std::cout << "Variable UmambiguousSpeed:"
          << targetListHeader->GetUmambiguousSpeed()
          << std::endl;
std::cout << "Variable AcquisitionStart:"
          << targetListHeader->GetAcquisitionStart()
          << std::endl;
// Getting members of  Target
for(auto& target : targetList)
{
    std::cout << "Variable Range:"
              << target->GetRange()
              << std::endl;
    std::cout << "Variable SpeedRadial:"
              << target->GetSpeedRadial()
              << std::endl;
    std::cout << "Variable AzimuthAngle:"
              << target->GetAzimuthAngle()
              << std::endl;
    std::cout << "Variable ElevationAngle:"
              << target->GetElevationAngle()
              << std::endl;
    std::cout << "Variable VarianceRange:"
              << target->GetVarianceRange()
              << std::endl;
    std::cout << "Variable VarianceSpeed:"
              << target->GetVarianceSpeed()
              << std::endl;
    std::cout << "Variable VarianceAzimuthAngle:"
              << target->GetVarianceAzimuthAngle()
              << std::endl;
    std::cout << "Variable VarianceElevationAngle:"
              << target->GetVarianceElevationAngle()
              << std::endl;
    std::cout << "Variable Rcs:"
              << target->GetRcs()
              << std::endl;
    std::cout << "Variable FalseAlarmProbability:"
              << target->GetFalseAlarmProbability()
              << std::endl;
    std::cout << "Variable Flags:"
              << target->GetFlags()
              << std::endl;
    std::cout << "Variable Power:"
              << target->GetPower()
              << std::endl;
    std::cout << "Variable Noise:"
              << target->GetNoise()
              << std::endl;
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
            std::cout << "Variable PeakIdx:"
                      << target->GetPeakIdx()
                      << std::endl;
        }

    }
        auto comDataStreamServ = com::master::umrr9f_t169_mse_v1_1_0::DataStreamServiceIface::Get();
        ClientId clientIdA = 1024; // client id from sensor a
        ClientId clientIdB = 1025; // client id from sensor b
        ReceiveComTargetListCallback callback =
                              std::bind(&ReceiveComTargetListClbk,
                              std::placeholders::_1,
                              std::placeholders::_2);
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetListReceiveCallback(clientIdA,
            callback)
        {
            std::cout << "Failed to register ComTargetList port callback" << std::endl;
        }
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetListReceiveCallback(clientIdB,
            callback))
        {
            std::cout << "Failed to register ComTargetList port callback" << std::endl;
        }
```

1.2 COMOBJECTLIST PORT

Description:
To receive the port called "ComObjectList" from a specific client, please use the following registration interface:

```
#include <umrr9f_t169_mse_v1_1_0/DataStreamServiceIface.h>

void ReceiveComObjectListClbk(IN std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::
    comobjectlist::ComObjectList> comObjectList, com::types::ClientId clientId)
{
        // Getting members of  ComObjectList
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comobjectlist::PortHeader> portHeader =
                        comObjectList->GetPortHeader();
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comobjectlist::ObjectListHeader>
        objectListHeader =
                        comObjectList->GetObjectListHeader();
    auto objectList = comObjectList->GetObjectList();
    // Getting members of  PortHeader
    std::cout << "Variable PortIdentifier:"
              << portHeader->GetPortIdentifier()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << portHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << portHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << portHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << portHeader->GetPortSize()
              << std::endl;
    std::cout << "Variable BodyEndianness:"
              << portHeader->GetBodyEndianness()
              << std::endl;
    std::cout << "Variable PortIndex:"
              << portHeader->GetPortIndex()
              << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
                << portHeader->GetHeaderVersionMajor()
                << std::endl;
        std::cout << "Variable HeaderVersionMinor:"
                << portHeader->GetHeaderVersionMinor()
                << std::endl;
        // Getting members of  ObjectListHeader
        std::cout << "Variable CycleTime:"
                << objectListHeader->GetCycleTime()
                << std::endl;
        std::cout << "Variable NumberOfObjects:"
                << objectListHeader->GetNumberOfObjects()
                << std::endl;
        std::cout << "Variable SelectedRefPoint:"
                << objectListHeader->GetSelectedRefPoint()
                << std::endl;
        std::cout << "Variable TimestampOfMeasurement:"
                << objectListHeader->GetTimestampOfMeasurement()
                << std::endl;
        // Getting members of  Object
        for(auto& object : objectList)
        {
            std::cout << "Variable PosX:"
                    << object->GetPosX()
                    << std::endl;
            std::cout << "Variable PosY:"
                    << object->GetPosY()
                    << std::endl;
            std::cout << "Variable PosZ:"
                    << object->GetPosZ()
                    << std::endl;
            std::cout << "Variable SpeedAbs:"
                    << object->GetSpeedAbs()
                    << std::endl;
            std::cout << "Variable Heading:"
                    << object->GetHeading()
                    << std::endl;
            std::cout << "Variable Length:"
                    << object->GetLength()
                    << std::endl;
            std::cout << "Variable Mileage:"
                    << object->GetMileage()
                    << std::endl;
            std::cout << "Variable Quality:"
                    << object->GetQuality()
                    << std::endl;
            std::cout << "Variable Acceleration:"
                    << object->GetAcceleration()
                    << std::endl;
            std::cout << "Variable ObjectId:"
                    << object->GetObjectId()
                    << std::endl;
            std::cout << "Variable IdleCycles:"
                    << object->GetIdleCycles()
                    << std::endl;
            std::cout << "Variable SplineIdx:"
                    << object->GetSplineIdx()
                    << std::endl;
            std::cout << "Variable ObjectClass:"
                    << object->GetObjectClass()
                    << std::endl;
            std::cout << "Variable Status:"
                    << object->GetStatus()
                    << std::endl;
        }

    }
        auto comDataStreamServ = com::master::umrr9f_t169_mse_v1_1_0::DataStreamServiceIface::Get();
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
    ClientId clientIdA = 1024; // client id from sensor a
    ClientId clientIdB = 1025; // client id from sensor b
    ReceiveComObjectListCallback callback =
                        std::bind(&ReceiveComObjectListClbk,
                        std::placeholders::_1,
                        std::placeholders::_2);
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComObjectListReceiveCallback(clientIdA,
        callback)
    {
        std::cout << "Failed to register ComObjectList port callback" << std::endl;
    }
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComObjectListReceiveCallback(clientIdB,
        callback))
    {
        std::cout << "Failed to register ComObjectList port callback" << std::endl;
    }
```

1.3 COMOBJECTBASELIST PORT

Description:
To receive the port called "ComObjectBaseList" from a specific client, please use the following registration interface:

```
#include <umrr9f_t169_mse_v1_1_0/DataStreamServiceIface.h>

void ReceiveComObjectBaseListClbk(IN std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::
    comobjectbaselist::ComObjectBaseList> comObjectBaseList, com::types::ClientId clientId)
{
        // Getting members of  ComObjectBaseList
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comobjectbaselist::PortHeader>
        portHeader =
                        comObjectBaseList->GetPortHeader();
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comobjectbaselist::
        ComObjectBaseListHeader> comObjectBaseListHeader =
                        comObjectBaseList->GetComObjectBaseListHeader();
    auto objectList = comObjectBaseList->GetObjectList();
    // Getting members of  PortHeader
    std::cout << "Variable PortIdentifier:"
              << portHeader->GetPortIdentifier()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << portHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << portHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << portHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << portHeader->GetPortSize()
              << std::endl;
    std::cout << "Variable BodyEndianness:"
              << portHeader->GetBodyEndianness()
              << std::endl;
    std::cout << "Variable PortIndex:"
              << portHeader->GetPortIndex()
              << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
              << portHeader->GetHeaderVersionMajor()
              << std::endl;
    std::cout << "Variable HeaderVersionMinor:"
              << portHeader->GetHeaderVersionMinor()
              << std::endl;
    // Getting members of  ComObjectBaseListHeader
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
        std::cout << "Variable Internal1:"
                  << comObjectBaseListHeader->GetInternal1()
                  << std::endl;
        std::cout << "Variable Speed:"
                  << comObjectBaseListHeader->GetSpeed()
                  << std::endl;
        std::cout << "Variable SpeedQuality:"
                  << comObjectBaseListHeader->GetSpeedQuality()
                  << std::endl;
        std::cout << "Variable YawRate:"
                  << comObjectBaseListHeader->GetYawRate()
                  << std::endl;
        std::cout << "Variable YawRateQuality:"
                  << comObjectBaseListHeader->GetYawRateQuality()
                  << std::endl;
        std::cout << "Variable DynamicSource:"
                  << comObjectBaseListHeader->GetDynamicSource()
                  << std::endl;
        std::cout << "Variable Internal2:"
                  << comObjectBaseListHeader->GetInternal2()
                  << std::endl;
        std::cout << "Variable NoOfObjects:"
                  << comObjectBaseListHeader->GetNoOfObjects()
                  << std::endl;
        std::cout << "Variable CycleDuration:"
                  << comObjectBaseListHeader->GetCycleDuration()
                  << std::endl;
        std::cout << "Variable CycleCount:"
                  << comObjectBaseListHeader->GetCycleCount()
                  << std::endl;
        // Getting members of  Object
        for(auto& object : objectList)
        {
            std::cout << "Variable XPoint1:"
                      << object->GetXPoint1()
                      << std::endl;
            std::cout << "Variable YPoint1:"
                      << object->GetYPoint1()
                      << std::endl;
            std::cout << "Variable SpeedAbs:"
                      << object->GetSpeedAbs()
                      << std::endl;
            std::cout << "Variable HeadingDeg:"
                      << object->GetHeadingDeg()
                      << std::endl;
            std::cout << "Variable ObjectLen:"
                      << object->GetObjectLen()
                      << std::endl;
            std::cout << "Variable Quality:"
                      << object->GetQuality()
                      << std::endl;
            std::cout << "Variable Acceleration:"
                      << object->GetAcceleration()
                      << std::endl;
            std::cout << "Variable ZPoint1:"
                      << object->GetZPoint1()
                      << std::endl;
            std::cout << "Variable ObjectId:"
                      << object->GetObjectId()
                      << std::endl;
        }

    }
        auto comDataStreamServ = com::master::umrr9f_t169_mse_v1_1_0::DataStreamServiceIface::Get();
        ClientId clientIdA = 1024; // client id from sensor a
        ClientId clientIdB = 1025; // client id from sensor b
        ReceiveComObjectBaseListCallback callback =
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
                        std::bind(&ReceiveComObjectBaseListClbk,
                        std::placeholders::_1,
                        std::placeholders::_2);
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComObjectBaseListReceiveCallback(clientIdA,
        callback)
    {
        std::cout << "Failed to register ComObjectBaseList port callback" << std::endl;
    }
    if(ERROR_CODE_OK != comDataStreamServ->RegisterComObjectBaseListReceiveCallback(clientIdB,
        callback))
    {
        std::cout << "Failed to register ComObjectBaseList port callback" << std::endl;
    }
```

1.4 COMTARGETBASELIST PORT

Description:
To receive the port called "ComTargetBaseList" from a specific client, please use the following registration interface:

```
#include <umrr9f_t169_mse_v1_1_0/DataStreamServiceIface.h>

void ReceiveComTargetBaseListClbk(IN std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::
    comtargetbaselist::ComTargetBaseList> comTargetBaseList, com::types::ClientId clientId)
{
        // Getting members of  ComTargetBaseList
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comtargetbaselist::PortHeader>
        portHeader =
                        comTargetBaseList->GetPortHeader();
    std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::comtargetbaselist::TargetListHeader>
        targetListHeader =
                        comTargetBaseList->GetTargetListHeader();
    auto targetList = comTargetBaseList->GetTargetList();
    // Getting members of  PortHeader
    std::cout << "Variable PortIdentifier:"
              << portHeader->GetPortIdentifier()
              << std::endl;
    std::cout << "Variable PortVersionMajor:"
              << portHeader->GetPortVersionMajor()
              << std::endl;
    std::cout << "Variable PortVersionMinor:"
              << portHeader->GetPortVersionMinor()
              << std::endl;
    std::cout << "Variable Timestamp:"
              << portHeader->GetTimestamp()
              << std::endl;
    std::cout << "Variable PortSize:"
              << portHeader->GetPortSize()
              << std::endl;
    std::cout << "Variable BodyEndianness:"
              << portHeader->GetBodyEndianness()
              << std::endl;
    std::cout << "Variable PortIndex:"
              << portHeader->GetPortIndex()
              << std::endl;
    std::cout << "Variable HeaderVersionMajor:"
              << portHeader->GetHeaderVersionMajor()
              << std::endl;
    std::cout << "Variable HeaderVersionMinor:"
              << portHeader->GetHeaderVersionMinor()
              << std::endl;
    // Getting members of  TargetListHeader
    std::cout << "Variable CycleDuration:"
              << targetListHeader->GetCycleDuration()
              << std::endl;
```


Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
        std::cout << "Variable NumberOfTargets:"
                  << targetListHeader->GetNumberOfTargets()
                  << std::endl;
        std::cout << "Variable CycleCount:"
                  << targetListHeader->GetCycleCount()
                  << std::endl;
        std::cout << "Variable AcquisitionSetup:"
                  << targetListHeader->GetAcquisitionSetup()
                  << std::endl;
        std::cout << "Variable TimeStamp:"
                  << targetListHeader->GetTimeStamp()
                  << std::endl;
        std::cout << "Variable AcqTimeStampFraction:"
                  << targetListHeader->GetAcqTimeStampFraction()
                  << std::endl;
        // Getting members of  Target
        for(auto& target : targetList)
        {
            std::cout << "Variable Range:"
                      << target->GetRange()
                      << std::endl;
            std::cout << "Variable SpeedRadial:"
                      << target->GetSpeedRadial()
                      << std::endl;
            std::cout << "Variable AzimuthAngle:"
                      << target->GetAzimuthAngle()
                      << std::endl;
            std::cout << "Variable ElevationAngle:"
                      << target->GetElevationAngle()
                      << std::endl;
            std::cout << "Variable RCS:"
                      << target->GetRCS()
                      << std::endl;
            std::cout << "Variable SignalLevel:"
                      << target->GetSignalLevel()
                      << std::endl;
            std::cout << "Variable Noise:"
                      << target->GetNoise()
                      << std::endl;
        }

    }
        auto comDataStreamServ = com::master::umrr9f_t169_mse_v1_1_0::DataStreamServiceIface::Get();
        ClientId clientIdA = 1024; // client id from sensor a
        ClientId clientIdB = 1025; // client id from sensor b
        ReceiveComTargetBaseListCallback callback =
                                std::bind(&ReceiveComTargetBaseListClbk,
                                std::placeholders::_1,
                                std::placeholders::_2);
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetBaseListReceiveCallback(clientIdA,
            callback)
        {
            std::cout << "Failed to register ComTargetBaseList port callback" << std::endl;
        }
        if(ERROR_CODE_OK != comDataStreamServ->RegisterComTargetBaseListReceiveCallback(clientIdB,
            callback))
        {
            std::cout << "Failed to register ComTargetBaseList port callback" << std::endl;
        }
```

For a more detailed API description, please see Appendix A.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

2 LEGAL DISCLAIMER NOTICE

All products, product specifications and data in this document may be subject to change without notice to improve reliability, function or otherwise. Not all products and/or product features may be available in all countries and regions. For legal reasons features may be deleted from products or smartmicro may refuse to offer products. Statements, technical information and recommendations contained herein are believed to be accurate as of the stated date. smartmicro disclaims any and all liability for any errors, inaccuracies or incompleteness contained in this document or in any other disclosure relating to the product.

To the extent permitted by applicable law, smartmicro disclaims (i) any and all liability arising out of the application or use of the product or the data contained herein, (ii) any and all liability of damages exceeding direct damages, including - without limitation - indirect, consequential or incidental damages, and (iii) any and all implied warranties, including warranties of the suitability of the product for particular purposes.

Statements regarding the suitability of products for certain types of applications are based on smartmicro's knowledge of typical requirements that are often placed on smartmicro products in generic/general applications. Statements about the suitability of products for a particular/specific application, however, are not binding. It is the customer's/user's responsibility to validate that the product with the specifications described is suitable for use in the particular/specific application. Parameters and the performance of products may deviate from statements made herein due to particular/specific applications and/or surroundings. Therefore, it is important that the customer/user has thoroughly tested the products and has understood the performance and limitations of the products before installing them for final applications or before their commercialization. Although products are well optimized to be used for the intended applications stated, it must also be understood by the customer/user that the detection probability may not be 100% and that the false alarm rate may not be zero.

The information provided, relates only to the specifically designated product and may not be applicable when the product is used in combination with other materials or in any process not defined herein. All operating parameters, including typical parameters, must be validated for each application by the customer's/user's technical experts. Customers using or selling smartmicro products for use in an application which is not expressly indicated do so at their own risk. This document does not expand or otherwise modify smartmicro's terms and conditions of purchase, including but not being limited to the warranty. Except as expressly indicated in writing by smartmicro, the products are not designed for use in medical, lifesaving or life-sustaining applications or for any other application in which the failure of the product could result in personal injury or death.

No license expressed or implied, by estoppel or otherwise, to any intellectual property rights is granted by this document or by any conduct of smartmicro. Product names and markings noted herein may be trademarks of their respective owners.

Please note that the application of the product may be subject to standards or other regulations that may vary from country to country. smartmicro does not guarantee that the use of products in the applications described herein will comply with such regulations in any country. It is the customer's/user's responsibility to ensure that the use and incorporation of products comply with regulatory requirements of their markets.

If any provision of this disclaimer is, or is found to be, void or unenforceable under applicable law, it will not affect the validity or enforceability of the other provisions of this disclaimer.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A COMMUNICATION DATA SERVICE API

A **ComTargetList**
Description: Target list port for automotive standard products.
The object ComTargetList provides the following APIs:

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::PortHeader> GetPortHeader() const;
```

Returns pointer to PortHeader object, whose access functions are described below:
A **PortHeader**
Description: smartmicro Generic Port Header
The object PortHeader provides the following APIs:

```
uint32_t GetPortIdentifier() const;
```

Returns value of PortIdentifier of uint32_t data type.
PortIdentifier - Unique port identifier

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major port version

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor port version

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - A timestamp

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Port size (includes this header)

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Port body endianness[1:big endian][2:little endian]

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - To identify multiple instances of the same port

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Generic port header major version

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Generic port header minor version

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::TargetListHeader> GetTargetListHeader()
    const;
```

Returns pointer to TargetListHeader object, whose access functions are described below:
A **TargetListHeader**
Description: Static header of the target list
The object TargetListHeader provides the following APIs:

```
float32_t GetCycleDuration() const;
```

Returns value of CycleDuration of float32_t data type.
CycleDuration - Cycle Time of the radar. Cycle time could vary depending on the mode of operation of the radar.

```
uint8_t GetNumberOfTargets() const;
```

Returns value of NumberOfTargets of uint8_t data type.
NumberOfTargets - Number of the targets detected by the radar.

```
uint32_t GetCycleCount() const;
```

Returns value of CycleCount of uint32_t data type.
CycleCount - Keeps a count of the total cycle durations that the radar has undergone since reboot starting from zero.

```
uint8_t GetAcquisitionSetup() const;
```

Returns value of AcquisitionSetup of uint8_t data type.
AcquisitionSetup - Feedback channel of used Acquisition setup

```
uint32_t GetTimeStamp() const;
```

Returns value of TimeStamp of uint32_t data type.
TimeStamp - TimeStamp in seconds at which the data acquisition(radar ramps) starts.

```
float32_t GetAcqTimeStampFraction() const;
```

Returns value of AcqTimeStampFraction of float32_t data type.
AcqTimeStampFraction - Timestamp(64Bit) is divided into two fields, first the timestamp(s) and second(AcqTimeStampFraction) which is the fraction part of the timestamp(s).

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
const std::vector<std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::Target>>&
    GetTargetList() const;
```

Returns pointer to array of Target objects, whose access functions are described below:
A **Target**
Description: Represent a single target
The object Target provides the following APIs:

```
float32_t GetRange() const;
```

Returns value of Range of float32_t data type.
Range - Range of the detected target.

```
float32_t GetSpeedRadial() const;
```

Returns value of SpeedRadial of float32_t data type.
SpeedRadial - Radial speed of the detected target.

```
float32_t GetAzimuthAngle() const;
```

Returns value of AzimuthAngle of float32_t data type.
AzimuthAngle - Azimuth angle of the target

```
float32_t GetElevationAngle() const;
```

Returns value of ElevationAngle of float32_t data type.
ElevationAngle - Elevation angle of the target

```
float32_t GetRCS() const;
```

Returns value of RCS of float32_t data type.
RCS - The radar cross section of the target in decibels.

```
float32_t GetSignalLevel() const;
```

Returns value of SignalLevel of float32_t data type.
SignalLevel - Power of the detected target in decibels.

```
float32_t GetNoise() const;
```

Returns value of Noise of float32_t data type.
Noise - Noise of the detected target in decibels.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A **ComObjectList**
Description: Object list port for automotive standard products.
The object ComObjectList provides the following APIs:

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::PortHeader> GetPortHeader() const;
```

Returns pointer to PortHeader object, whose access functions are described below:
A **PortHeader**
Description: smartmicro Generic Port Header
The object PortHeader provides the following APIs:

```
uint32_t GetPortIdentifier() const;
```

Returns value of PortIdentifier of uint32_t data type.
PortIdentifier - Unique port identifier

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major port version

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor port version

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - A timestamp

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Port size (includes this header)

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Port body endianness[1:big endian][2:little endian]

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - To identify multiple instances of the same port

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Generic port header major version

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Generic port header minor version

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::ObjectListHeader> GetObjectListHeader()
    const;
```

Returns pointer to ObjectListHeader object, whose access functions are described below:
A **ObjectListHeader**
Description: Static header of the object list
The object ObjectListHeader provides the following APIs:

```
float32_t GetCycleTime() const;
```

Returns value of CycleTime of float32_t data type.
CycleTime - Cycle time

```
uint16_t GetNumberOfObjects() const;
```

Returns value of NumberOfObjects of uint16_t data type.
NumberOfObjects - Number of objects in the current object list

```
uint8_t GetSelectedRefPoint() const;
```

Returns value of SelectedRefPoint of uint8_t data type.
SelectedRefPoint - Selected reference point of the objects.

```
uint64_t GetTimestampOfMeasurement() const;
```

Returns value of TimestampOfMeasurement of uint64_t data type.
TimestampOfMeasurement - Start of radar acquisition for object list (NTP coded)

```
const std::vector<std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::Object>>&
    GetObjectList() const;
```

Returns pointer to array of Object objects, whose access functions are described below:
A **Object**
Description: Represents a single object.
The object Object provides the following APIs:

```
float32_t GetXPoint1() const;
```

Returns value of XPoint1 of float32_t data type.
XPoint1 - X-component of the position, depends on selected reference point.

```
float32_t GetYPoint1() const;
```

Returns value of YPoint1 of float32_t data type.
YPoint1 - Y-component of the position, depends on selected reference point.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
float32_t GetSpeedAbs() const;
```

Returns value of SpeedAbs of float32_t data type.
SpeedAbs - Absolute object speed.

```
float32_t GetHeadingDeg() const;
```

Returns value of HeadingDeg of float32_t data type.
HeadingDeg - Heading of the object.

```
float32_t GetObjectLen() const;
```

Returns value of ObjectLen of float32_t data type.
ObjectLen - Length of the object.

```
float32_t GetQuality() const;
```

Returns value of Quality of float32_t data type.
Quality - Quality of the object.

```
float32_t GetAcceleration() const;
```

Returns value of Acceleration of float32_t data type.
Acceleration - Object acceleration.

```
float32_t GetZPoint1() const;
```

Returns value of ZPoint1 of float32_t data type.
ZPoint1 - Z-component of the position, depends on selected reference point.

```
uint16_t GetObjectId() const;
```

Returns value of ObjectId of uint16_t data type.
ObjectId - Unique tag number for identification of the object.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A **ComObjectBaseList**
Description: Base object list port for automotive.
The object ComObjectBaseList provides the following APIs:

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::PortHeader> GetPortHeader() const;
```

Returns pointer to PortHeader object, whose access functions are described below:
A **PortHeader**
Description: smartmicro Generic Port Header
The object PortHeader provides the following APIs:

```
uint32_t GetPortIdentifier() const;
```

Returns value of PortIdentifier of uint32_t data type.
PortIdentifier - Unique port identifier

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major port version

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor port version

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - A timestamp

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Port size (includes this header)

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Port body endianness[1:big endian][2:little endian]

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - To identify multiple instances of the same port

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Generic port header major version

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Generic port header minor version

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::ComObjectBaseListHeader>
    GetComObjectBaseListHeader() const;
```

Returns pointer to ComObjectBaseListHeader object, whose access functions are described below:
A **ComObjectBaseListHeader**
Description: Static header of the object list.
The object ComObjectBaseListHeader provides the following APIs:

```
uint64_t GetInternal1() const;
```

Returns value of Internal1 of uint64_t data type.
Internal1 - Internal reserved field.

```
float32_t GetSpeed() const;
```

Returns value of Speed of float32_t data type.
Speed - Ego Speed. Source dependant on dynamics source: 0 = internal ego speed estimation. 1,2 or 3 - ego speed coming from an external source via ETH or CAN.

```
float32_t GetSpeedQuality() const;
```

Returns value of SpeedQuality of float32_t data type.
SpeedQuality - Value that ranges from 0-1, indicating how well supported the current input of the ego speed is. Can behave differently, depending on the source.

```
float32_t GetYawRate() const;
```

Returns value of YawRate of float32_t data type.
YawRate - Yaw rate of the ego vehicle. Is 0 when the dynamics source == 0, as there is no ego yaw rate estimation. Can be != 0 if it is given by an external source.

```
float32_t GetYawRateQuality() const;
```

Returns value of YawRateQuality of float32_t data type.
YawRateQuality - Value that ranges from 0-1, indicating how well supported the current input of the ego yaw rate is.

```
uint8_t GetDynamicSource() const;
```

Returns value of DynamicSource of uint8_t data type.
DynamicSource - Describing the source of the ego dynamics information.[0:Internal ego speed estimation.][1:External dynamics input via ETH.][2:External dynamics input via CAN.][3:Internal ego speed estimation + optional external yaw rate via CAN.]

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint64_t GetInternal2() const;
```

Returns value of Internal2 of uint64_t data type.
Internal2 - Internal reserved field.

```
uint16_t GetNoOfObjects() const;
```

Returns value of NoOfObjects of uint16_t data type.
NoOfObjects - Number of objects in the current object list.

```
float32_t GetCycleDuration() const;
```

Returns value of CycleDuration of float32_t data type.
CycleDuration - Duration of the current radar cycle in seconds.

```
uint32_t GetCycleCount() const;
```

Returns value of CycleCount of uint32_t data type.
CycleCount - Keeps a count of the total cycle durations that the radar has undergone since reboot starting from zero.

```
const std::vector<std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::Object>>&
    GetObjectList() const;
```

Returns pointer to array of Object objects, whose access functions are described below:
A **Object**
Description: Represents a single object.
The object Object provides the following APIs:

```
float32_t GetXPoint1() const;
```

Returns value of XPoint1 of float32_t data type.
XPoint1 - X-component of the position, depends on selected reference point.

```
float32_t GetYPoint1() const;
```

Returns value of YPoint1 of float32_t data type.
YPoint1 - Y-component of the position, depends on selected reference point.

```
float32_t GetSpeedAbs() const;
```

Returns value of SpeedAbs of float32_t data type.
SpeedAbs - Absolute object speed.

```
float32_t GetHeadingDeg() const;
```

Returns value of HeadingDeg of float32_t data type.
HeadingDeg - Heading of the object.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
float32_t GetObjectLen() const;
```

Returns value of ObjectLen of float32_t data type.
ObjectLen - Length of the object.

```
float32_t GetQuality() const;
```

Returns value of Quality of float32_t data type.
Quality - Quality of the object.

```
float32_t GetAcceleration() const;
```

Returns value of Acceleration of float32_t data type.
Acceleration - Object acceleration.

```
float32_t GetZPoint1() const;
```

Returns value of ZPoint1 of float32_t data type.
ZPoint1 - Z-component of the position, depends on selected reference point.

```
uint16_t GetObjectId() const;
```

Returns value of ObjectId of uint16_t data type.
ObjectId - Unique tag number for identification of the object.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

A **ComTargetBaseList**
Description: Base target list port for automotive
The object ComTargetBaseList provides the following APIs:

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::PortHeader> GetPortHeader() const;
```

Returns pointer to PortHeader object, whose access functions are described below:
A **PortHeader**
Description: smartmicro Generic Port Header
The object PortHeader provides the following APIs:

```
uint32_t GetPortIdentifier() const;
```

Returns value of PortIdentifier of uint32_t data type.
PortIdentifier - Unique port identifier

```
int16_t GetPortVersionMajor() const;
```

Returns value of PortVersionMajor of int16_t data type.
PortVersionMajor - Major port version

```
int16_t GetPortVersionMinor() const;
```

Returns value of PortVersionMinor of int16_t data type.
PortVersionMinor - Minor port version

```
uint64_t GetTimestamp() const;
```

Returns value of Timestamp of uint64_t data type.
Timestamp - A timestamp

```
uint32_t GetPortSize() const;
```

Returns value of PortSize of uint32_t data type.
PortSize - Port size (includes this header)

```
uint8_t GetBodyEndianness() const;
```

Returns value of BodyEndianness of uint8_t data type.
BodyEndianness - Port body endianness[1:big endian][2:little endian]

```
uint8_t GetPortIndex() const;
```

Returns value of PortIndex of uint8_t data type.
PortIndex - To identify multiple instances of the same port

```
uint8_t GetHeaderVersionMajor() const;
```

Returns value of HeaderVersionMajor of uint8_t data type.
HeaderVersionMajor - Generic port header major version

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
uint8_t GetHeaderVersionMinor() const;
```

Returns value of HeaderVersionMinor of uint8_t data type.
HeaderVersionMinor - Generic port header minor version

```
std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::TargetListHeader> GetTargetListHeader()
    const;
```

Returns pointer to TargetListHeader object, whose access functions are described below:
A **TargetListHeader**
Description: Static header of the target list
The object TargetListHeader provides the following APIs:

```
float32_t GetCycleDuration() const;
```

Returns value of CycleDuration of float32_t data type.
CycleDuration - Cycle Time of the radar. Cycle time could vary depending on the mode of operation of the radar.

```
uint8_t GetNumberOfTargets() const;
```

Returns value of NumberOfTargets of uint8_t data type.
NumberOfTargets - Number of the targets detected by the radar.

```
uint32_t GetCycleCount() const;
```

Returns value of CycleCount of uint32_t data type.
CycleCount - Keeps a count of the total cycle durations that the radar has undergone since reboot starting from zero.

```
uint8_t GetAcquisitionSetup() const;
```

Returns value of AcquisitionSetup of uint8_t data type.
AcquisitionSetup - Feedback channel of used Acquisition setup

```
uint32_t GetTimeStamp() const;
```

Returns value of TimeStamp of uint32_t data type.
TimeStamp - TimeStamp in seconds at which the data acquisition(radar ramps) starts.

```
float32_t GetAcqTimeStampFraction() const;
```

Returns value of AcqTimeStampFraction of float32_t data type.
AcqTimeStampFraction - Timestamp(64Bit) is divided into two fields, first the timestamp(s) and second(AcqTimeStampFraction) which is the fraction part of the timestamp(s).

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

```
const std::vector<std::shared_ptr<com::master::umrr9f_t169_mse_v1_1_0::Target>>&
    GetTargetList() const;
```

Returns pointer to array of Target objects, whose access functions are described below:
A **Target**
Description: Represent a single target
The object Target provides the following APIs:

```
float32_t GetRange() const;
```

Returns value of Range of float32_t data type.
Range - Range of the detected target.

```
float32_t GetSpeedRadial() const;
```

Returns value of SpeedRadial of float32_t data type.
SpeedRadial - Radial speed of the detected target.

```
float32_t GetAzimuthAngle() const;
```

Returns value of AzimuthAngle of float32_t data type.
AzimuthAngle - Azimuth angle of the target

```
float32_t GetElevationAngle() const;
```

Returns value of ElevationAngle of float32_t data type.
ElevationAngle - Elevation angle of the target

```
float32_t GetRCS() const;
```

Returns value of RCS of float32_t data type.
RCS - The radar cross section of the target in decibels.

```
float32_t GetSignalLevel() const;
```

Returns value of SignalLevel of float32_t data type.
SignalLevel - Power of the detected target in decibels.

```
float32_t GetNoise() const;
```

Returns value of Noise of float32_t data type.
Noise - Noise of the detected target in decibels.

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

B USER INTERFACE INSTRUCTIONS UMRR9F T169 MSE VERSION 1.1.0

B.1 Parameter Section auto_interface_0dim

Automotive user interface 0dimensional parameters

Parameter Name	tx_antenna_idx
Description	Index of Transmit Antenna
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	center_frequency_idx
Description	Index of center frequency
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	2

Parameter Name	frequency_sweep_idx
Description	Index of sweep (0=226MHz)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	0

Parameter Name	prf_selector_manual
Description	0 = PRF switching active, 1 = PRF index given in prf_selector_index is used
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	1

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	prf_set_selector
Description	Select PRF set index (only one set supported)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	0

Parameter Name	prf_manual_value_idx
Description	In manual PRF mode only: use nth element of selected set
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	2

Parameter Name	detection_sensitivity
Description	Detection sensitivity: 0=low, 1=normal, 2=high
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	2

Parameter Name	tv_min_speed_sweep_idx_0
Description	Target Validation: minimum speed of target at first sweep
Data Type	f32
Dimensions	None
Access	RW
Default	-150.0
Min	-150.0
Max	150.0

Parameter Name	tv_max_speed_sweep_idx_0
Description	Target Validation: maximum speed of target at first sweep
Data Type	f32
Dimensions	None
Access	RW
Default	150.0
Min	-150.0
Max	150.0

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	azimuthal_angle_org
Description	[rad] Original Orientation of the sensor in Cartesian coordinate system in X-Y plane configured during setup, valid interval (-pi, pi)
Data Type	f32
Dimensions	None
Access	RW
Default	0
Min	-3.14159265
Max	3.14159265

Parameter Name	elevation_angle_org
Description	[rad] Original Orientation of the sensor in Cartesian coordinate system in X-Z plane configured during setup, valid interval [-pi/2, pi/2]
Data Type	f32
Dimensions	None
Access	RW
Default	0
Min	-1.57079642
Max	1.57079642

Parameter Name	output_control_target_list_can
Description	send raw targets via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_object_list_can
Description	send objects via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_target_list_eth
Description	send raw targets via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	output_control_object_list_eth
Description	send objects via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_crash_barrier_can
Description	send crash barrier data via CAN, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_dynamics_eth
Description	send dynamics data via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	output_control_diagnostic_eth
Description	send diagnostic port via Ethernet, 0 = disabled, 1 = enabled
Data Type	u8
Dimensions	None
Access	RW
Default	1
Min	0
Max	1

Parameter Name	ip_source_address
Description	IP source address (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	3232238347

Parameter Name	subnet_mask
Description	Subnet mask (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	4294967040

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Parameter Name	ip_dest_address
Description	IP destination address (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	3232238353

Parameter Name	ip_dest_port
Description	IP destination port
Data Type	u16
Dimensions	None
Access	RW
Default	55555

Parameter Name	dynamics_source_switch
Description	0: Dynamics Data Source: dynSpdYawEst Port (ALGO), 1: Ethernet (dynamicsExt port), 2: Dynamics Data Source: pdrv Port (CAN) only speed and yawrate, 3: Speed (ALGO), Yaw Rate (extern CAN)
Data Type	u8
Dimensions	None
Access	RW
Default	0
Min	0
Max	3

Parameter Name	mc_dest_address
Description	Multicast destination address for Alive (32bit)
Data Type	u32
Dimensions	None
Access	RW
Default	4019191808
Min	3758096384
Max	4026531839

Parameter Name	mc_port
Description	Multicast destination port for Alive
Data Type	u16
Dimensions	None
Access	RW
Default	60000
Min	1

B.2 Status Section auto_interface

customer status section

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Status Name	auto_interface_version_major
Description	Automotive interface version major. Increased, if new version is not totally backward compatible.
Data Type	u32
Dimensions	None
Access	R

Status Name	auto_interface_version_minor
Description	Automotive interface version minor. Increased, if parameters or statuses are changed or added. The new version is still backward compatible.
Data Type	u32
Dimensions	None
Access	R

Status Name	sw_generation
Description	Software Version generation
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_major
Description	Software Version major
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_minor
Description	Software Version minor
Data Type	u16
Dimensions	None
Access	R

Status Name	sw_version_patch
Description	Software Version patch
Data Type	u16
Dimensions	None
Access	R

Status Name	customer_id
Description	Customer Identifier
Data Type	u16
Dimensions	None
Access	R

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Status Name	product_serial
Description	32Bit product id serial
Data Type	u32
Dimensions	None
Access	R

Status Name	product_gen
Description	product generation
Data Type	u32
Dimensions	None
Access	R

Status Name	product_mod_high
Description	product modification high
Data Type	u32
Dimensions	None
Access	R

Status Name	product_mod_low
Description	product modification low
Data Type	u32
Dimensions	None
Access	R

Status Name	product_rev
Description	product revision
Data Type	u32
Dimensions	None
Access	R

Status Name	crc32_val
Description	32 bit crc check value
Data Type	u32
Dimensions	None
Access	R

Status Name	crc_calc_done
Description	flag to indicate if crc calculation is done
Data Type	u32
Dimensions	None
Access	R

B.3 Command Section auto_interface_command

Maintain compatible section 1000 commands

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.

Command Name	comp_fsm_core0_opmode
Description	Select top level FSM operation mode (3078.1)

Command Name	comp_sw_crc_check
Description	Starts the software crc check

Command Name	comp_eeprom_ctrl_factory_reset
Description	Performs factory reset (3102.4)

Command Name	comp_sensor_reset
Description	Reset command which starts from BIOS (if available) or bootloader (3074.1)

Command Name	comp_pdi_requestor_can
Description	Send PDI data to client (3076.1)

Command Name	comp_eeprom_ctrl_save_param_sec
Description	Save the parameter inside the EEPROM. (3102.3)

Command Name	comp_eeprom_ctrl_reset_param_sec
Description	Restore default values in RAM. EEPROM content is not changed. (3102.2)

Command Name	comp_eeprom_ctrl_default_param_sec
Description	Restore default values in RAM and EEPROM. (3102.1)

Command Name	comp_timebase_set_seconds_val
Description	Set SECONDS value of NTP UTC timestamp

Command Name	comp_timebase_set_frac_seconds_val
Description	Set FRACTION_SECONDS value of NTP UTC timestamp

Proprietary - This document may be subject to change without notice. The information shall remain the exclusive property of s.m.s, smart microwave sensors GmbH.